CAUSAL MAP GARDEN

# Task 2 – Coding with AI

## Contents

# Intro – auto-coding with AI

Chapter contents.

15 Oct 2025

Causal coding is fascinating but can take a lot of time. Using AI to help you is pretty easy, but there are still quite a few things to think about, as we will explain in this chapter.

PAGES IN THIS CHAPTER

# AI coding overview

2 May 2026

## Summary

This page is an overview of the choices you face when doing AI coding (qualitative causal coding with AI) inside the Causal Map app.

- For step-by-step app docs see AI coding
- For the broader argument about why we use AI as a low-level coder rather than as a black box, see AI in evaluation actually show your working! and Just add rigour Three do's and don'ts.

## Workflow

Someone arrives with a stack of documents and asks: can you code these? It depends first on various parallel setup decisions. After coding, you face a separate decision about how, or whether, to recode the labels.

## Decisions before coding

### *Random-sample coding strategy with bigger corpora*

We will usually test and improve the instruction(s) on just a small random sample of the material. It's important that it is random, so that you don't waste time fitting an instruction to just one type of material which is maybe not typical for the whole corpus of text. The Causal Map app allows you to select random samples overall or, even better, random samples from each of relevant groups like women and men.

If you have more than around 100 pages, work on a sample first. The app has features to take a random sample of sources, or a stratified random sample within source groups. The AI coding panel also offers a sample-only run (see AI coding). Try the sample, review, adjust your strategy, perhaps update the codebook, then run a larger sample. With 1000 pages you might code 100, adjust, code another 300 (deleting the first attempt), and if that looks good, finish with the remaining 700.

#### Using additional iterations

When coding with AI, we often use one or more additional AI iterations, e.g.

- Accuracy: to check that the coded links have been coded correctly (and if necessary, then amend/delete them)
  - according to the original rules
  - according to new/additional criteria
- Coverage[1]: to check that links which should have been coded were indeed identified and if not to add them. Each additional iteration sends the original text and instructions and the results back to the AI, i.e. the "conversation" to date, along with additional specific instructions for this iteration. So if you add two additional iterations, the results will take altogether three times as long (and cost three times as much) as the same task without iterations.

We often find that our time is better spent improving the original coding instruction than adding additional iterations.

#### Improving the instructions

The most important trick of all is not just to apply a generic prompt / coding instruction, with or without additional iterations, but in initial testing to examine the results, work out specifically the reasons for any problems with accuracy or coverage, and to **tweak the instruction and try again**, repeating until you are satisfied. We usually do this before adding any additional iterations and then if the additional iteration(s) do improve the results, we check them as above to see if we can improve the *iteration* instruction.

When there is a lot of text to process (hundreds or thousands of pages of text) we will sometimes start off with a small sample as described above and then, once we are satisfied, check again on a larger sample before coding the entire corpus.

### *Codebook strategy: how free?*

You can start with a full codebook, a partial codebook, or nothing.

A *forced* codebook restricts the model to your labels (for example from a theory of change). If a causal claim mentions a factor not in the codebook, no link is coded.

Most often you provide a codebook but allow the model to invent labels when nothing fits. This is harder to manage: telling the model it *can* improvise seems to confuse it a bit, and codebook coverage drops.

Four codebook strategies:

1. Stick only to the codebook. Anything that doesn't match is dropped.
2. Stick to the codebook mostly, but let the AI code other things too. Tell it to flag the new ones with a tag like `[new]` or a trailing `*`, so you can find and review them later.
3. Compromise. Use only top-level labels from the codebook, but let the AI improvise the second part of a hierarchical label. See Hierarchical coding.]
4. Free coding

Related to the above...

## *Label style: In vivo or abstract*

When free coding, you can ask for *in vivo* labels (close to the original text) or for more abstract labels. There are many ways to instruct: "talk like a social scientist" or "talk like a local newspaper editor".

You can suggest to the AI to use a common “social sciences” vocabulary like:

- presence of resources
- lack of resources
- more/better income
- more/better motivation
- more/better support from peers

W3lcom3!

## *Label style: Using tags*

There's often more to a coding task than just labels. *Tags* are short bits of text in brackets attached to labels: `stressed patients (before surgery)`, `stressed patients (after surgery)`. Tags help you build up a system of labels from smaller parts.

## *Label style: Opposites*

See !Opposites and sentiment in AI coding and Opposites.

## *Label style: Hierarchical*

You can impose hierarchical labels even when free coding. See Hierarchical coding.

## *Columns*

We also often ask for *columns* unrelated to the labels, such as a sentiment assessment. Columns can be useful for any systematic attribute you can code consistently across most links. Note the difference: columns code attributes of *links*; tags code attributes of *factors*.

Asking the model to fill extra columns alongside coding is extra work for it. Expect either coverage (links found) or precision to drop.

### Columns: Sentiment

Zero-codebook coding usually leads on to magnetic soft recoding, and in embedding space `decrease in X` sits close to `increase in X`.

With an explicit codebook, you can get round this by (maybe) suggesting both sides of any factor likely to appear positively and negatively, e.g. `increased income` and `decreased income`.

When you don't specify a codebook, get the model to code sentiment for each link.

Sentiment is a kind of column.

A sentiment column lets you tell them apart.

Sentiment can also be interesting for other reasons.

## *Other things which are important in the prompt*

Often you want to tell the model what the work is for: the context, named entities, the audience. We *iterate* on the prompt, trying versions and comparing results.

### Context

...

### Named entities

We want the coding AI to recognise words or phrases or abbreviations specific to our project which are not general knowledge and also know that there may be different words for the same thing, e.g. different ways of talking about the same project or organisation; and we usually want it to then just use one preferred phrase even if there are alternatives in the text.

Our preferred phrase should normally be in the same language as the other labels we ask it to make.

## *Coding style: holistic or claim-by-claim?*

Two main approaches.

*Holistic*: we ask the model for an overall diagram of the causal network in the current chunk of text[2]. The model decides what the main links are, but we still ask for a quote behind each one.

Especially when using a holistic style, it can be useful to add a second iteration to “mop up” anything missed.

*Claim by claim*: we ask the model to find each individual causal link. Hundreds of tweaks and heuristics later, this style still struggles to tell a connected story even when the text contains one. Suppose the text supports A to B to C to D, but the model lazily codes A to B and C to D, using slightly different labels for B and C. Claim-by-claim coding only really works if you plan to recode afterwards: you hope a later recoding pass spots that B and C mean the same thing and rejoins the chain.

## *Model*

For relatively straightforward cases, newer or bigger models are not necessarily better for coding. We have had good results with Gemini Flash, for example.

## *How many iterations?*

Additional iterations can be useful:

- For checking and improving quality
- For increasing coverage (finding more actual causal claims)
- For adding additional information (e.g. more columns)

Multi-stage prompts (separated by `====`) let you split coding into sequential passes. The UI does this for you. Mechanics in !1010-Auto-coding and AI coding under "Prompt sections".

If you are asking the AI to provide substantial additional coding, such as adding more columns like, say, "significance" or "certainty" or to add a translation or some other text column, we recommend using one or more additional iterations for this because you don't want the AI to miss out or mis-code the actual links, which after all are the most important output of the whole process.

More advanced models are more capable of doing more things at the same time during coding.

## *Chunk and sample strategy*

If a single source is short, you can map the whole thing in one go. More often you have either much longer sources, which the app breaks into chunks for you, or many sources.

### Recall and precision

A pretty hard rule: the more text you give the model at once, the less dense its coding gets. One page might yield 20 links; 5 pages might also yield 20 links. Sometimes the 20 it picks out of 5 pages really are the most important, but this is not certain, and you are leaving the model to decide. Often you just want better recall, and that means smaller chunks. Don't give the model too much freedom to decide what counts as important.

### What does it select

Bigger chunks mean sparser coding. There is a "code everything" setting that does not each source at all: the effect depends on how long your sources are. Otherwise, work in smaller chunks. Aiming to pick up *every* causal claim is unrealistic, but the smaller the chunk, the more you'll catch.

## Recoding style

Once the first coding run exists, the next question is how to deal with overlapping labels. See Different kinds of coding and recoding

Recoding from scratch means arriving at a clean revised codebook and then recoding everything from the beginning. That's more expensive and slower, but usually gives better results than magnetically recoded labels alone (see Different kinds of coding and recoding).

- *Hard recoding*: revise the codebook and code again.
- *Links recoding*: use AI Answers / Links to recode links, with quote and context available.
- *Factors recoding*: use AI Answers / Factors to recode factor labels directly.
- *Soft recoding*: use clustering or magnetic labels as a softer recoding layer.

## Related

- chapter intro
- AI coding: app docs for the AI Coding panel
- AI answers panel: app docs for AI Answers
- !1010- Auto-coding: older detailed notes on the auto-coding prompt
- !Opposites and sentiment in AI coding
- ! Autoclustering with embeddings
- 
- AI in evaluation actually show your working!: the transparency argument
- Just add rigour Three do’s and don’ts: do's and don'ts for AI text analysis
- Causal mapping is easy to automate transparently, so is a great fit for scaling with AI

1. Also known as Recall ↩
2. Internally, we ask it for a Mermaid diagram then convert it into a links table. For some reason, LLMs often do a much better job of creating a connected network if you ask for a diagram than if you ask for a set of connected links ↩

# 🌻 Different kinds of coding and recoding

Assuming you're not coded with a fixed codebook, and assuming you're coding multiple sources or you've got texts which are longer than your selected chunk size, so you are breaking them up into multiple chunks, then you can expect to end up with very many labels, many of which overlap in meaning.

You can address this either with

- soft recoding or magnetic relabelling
- hard recoding: once we've done a fair amount of coding, or all the coding, we then group those labels either pre-clustering them using embeddings or and or applying an AI and or putting a human in the loop to get a list of labels for hard coding, or rather a system because you might have a smaller set of labels and additional tags or columns.

We haven't done the research to find out whether having say 10 labels and three tags is more or less efficient than the corresponding set of 60 labels.

And of course there's also the newer possibility of using the AI in the AI Answers feature to take an existing large set of coded labels and then recode them into a more compact set. This has also been discussed here Different kinds of coding and recoding

So you've basically got a lot of decisions to take as you work your way through the coding pathway. And it all depends on lots of different things like how long your text is, how many different documents you have. Oh, I didn't mention that in Causal Map we never, we do break down long source texts into smaller chunks but we never combine smaller source texts into larger chunks, so each source is always coded on its own. So if you don't have a fixed codebook then you'll always get many labels with overlapping meanings.

| | | | | | |
|---|---|---|---|---|---|
| | **Hard coding** | **Hard recoding** | **Links recoding** | **Factors recoding** | **Soft recoding** |
| Accuracy | Highest | ... | ... | ... | Lowest |
| Speed | Slowest | ... | ... | ... | Fastest |
| | | | | | |

| | Hard coding | Hard recoding | Links recoding | Factors recoding | Soft recoding |
|---|---|---|---|---|---|
| Manual | Just code manually | Make a copy of your file, delete links and start again | Edit manually in Links table or Map,<br>- or use search/replace in Links table | Edit manually in Factors table or Map,<br>- or use search/replace in Factors table<br>- or Bulk Edit | - |
| AI | Just code with AI, with/without a codebook | As above, or just put the switch "skip coded sources" to off | AI Answers / Links.<br>Writes into whichever Label Set is active in the toolbar. | AI Answers / Factors.<br>Writes into whichever Label Set is active in the toolbar. | Apply magnetic labels in Soft Recode filter |

What's the point of Links and Factors recoding? What's the difference?

- Soft recoding is only as good as the underlying embedding space, and it is never perfect.
- Hard recoding can take a long time, is expensive, and does not encourage experimentation
- With Links/Factors recoding, you can:
  - Recode just the currently filtered sources/links (or all links)
  - Recode into whichever Label Set is active. Pick `default` in the Label Set widget (toolbar, below the Sources bar) to write to the permanent cause/effect labels; create a named set such as `experiment1` and the AI writes to `cause_experiment1` / `effect_experiment1` instead. Flip between sets in the same widget to view either.
  - There is also another option Answers which is not about recoding; it is simply a way to send your links and/or factors data to an AI and getting a text answer.

But the main point is that rather than just hoping the magnetisation will work the way you want it to, you can do smart recoding as if you had an assistant to work through each label. For example you can say "Relabel everything which expresses a decrease or lack of something with a ~" or "Look at all these labels and tag each with `[Food]` or `[Health]"

.

- You can even bring other columns into play, for example citation count, source count etc.
- Links recoding is significantly more powerful because you can also include the actual Quote as well as both Cause and Effect. This means the AI can make its decision with a lot more context. So this is almost like recoding from scratch, but the original coding has already identified causal claims and all we have to do is relabel the labels with the same complete information about the claim.
- Be careful: it's tempting to say things like "Find 3-8 top-level factor labels which cover the meaning of all these labels and recode them with the new top-level labels", but remember the "Rows per call"

slider: with a large set of links and lots of quotes you will probably have to break up your work into multiple chunks, and each call may come up with different labels. In this case you could use the Answers mode (or the Cluster part of Soft Recode filter) first to develop some labels.